Integrating a SMT Solver based Local Search in Ant Colony Optimization for Solving RCMPSP

Mariam Gómez Sánchez
Departamento de Informática
Universidad Técnica Federico Santa María
Valparaíso, Chile
mggomez@jp.inf.utfsm.cl

Alejandro Fernández Gil
Departamento de Informática
Universidad Técnica Federico Santa María
Valparaíso, Chile
affernan@jp.inf.utfsm.cl

Carlos Castro
Departamento de Informática
Universidad Técnica Federico Santa María
Valparaíso, Chile
Carlos.Castro@inf.utfsm.cl

***Abstract*—The project scheduling problem has been widely studied given the practical importance it has and the complexity to find efficient solutions. In this work we address an extension of the same problem based on the Resource-Constrained Multi-Project Scheduling Problem, taking into account that each project has a set of own resources available for consumption. We propose a hybrid algorithm based on Ant Colony Optimization and a Local Search procedure based on Hill-Climbing First Improvement supported by an SMT Solver as a movement satisfaction verifier. Our approach was tested with a set of 12 instances belonging to the MPSPLIB datasets. We have obtained near-optimal solutions for several instances of the problem.**

***Index Terms*—Ant Colony Optimization, Combinatorial Optimization, Multi Project Scheduling Problem, Local Search, SMT Solver.**

I. Introduction

The Resource-Constrained Project Scheduling Problem (RCPSP) consists of the allocation of processing starting times of various activities that are part of a project and must follow a strict order of precedence. This class of problems represents one of the biggest challenges facing industries. RCPSP arises in the decade of 1960 [1] and is considered NP-Hard [2]. Pritsker et al. [3] addresses the Resource-Constrained Multi-Project Scheduling Problem (RCMPSP), where several projects must be executed simultaneously competing for the use of global resources. Confessore et al. [4] propose the Decentral Resource Constrained Multi-Project Scheduling Problem (DRCMPSP) with the particularity that each project has its own set of renewable resources managed by an intelligent agents and global resources are managed by an central intelligent agent that does not know the need of own resources in each project. The objective pursued in the problems described above is based on finding a priority and feasible schedule of resources for a set of activities in each project, minimizing mainly the total time of completion of all activities (`Makespan`) added to the time of delay (`Delay`) in each project.

In this work we focus on solving DRCMPSP, without taking into account the decentralized part of the problem, having only a decision maker, this is similar to solving an RCMPSP with global and local resource constraints. We propose an Ant Colony Optimization (ACO) [5] approach, a method of swarm intelligence, to solve this problem. Later, we apply a Local Search procedure based on Hill-Climbing (HC) First Improvement. As part of HC, we used Z3 [6], a SMT solver to support the process of verifying movement satisfaction and precedence of activities, formalizing the constraints as logical formulas. We have obtained near-optimal solutions for several instances of the problem.

The remainder of this paper is organized as follows. Section II describes the RCMPSP. Section III reviews related work. The proposal description with the integrating algorithms are given in Section IV. The experiments and results are showed in Sections V and VI. Finally, we present the conclusions and future work in Section VII.

II. Problem Description

The RCMPSP with global and local resource constraints considers a set of $\mathcal{P}$ projects that are scheduled simultaneously. Each p project has a set of A_p activities and a set of local renewable resources R_p to meet the demands of the activities belonging to A_p. Each a activity of a p project has a duration of d_{ap}, a set of successor activities S_{ap} and a set of resources necessary for its execution r_{ap}. Until the end of the execution of the a activity of the p project, activities belonging to S_{ap} cannot begin to be executed. For the start of the execution of the a activity of the p project, all resources r_{ap} must be available simultaneously in the defined quantities and this availability must remain uninterrupted until the end of the execution. There is a set of global renewable resources $\mathcal{G}$ shared by all projects. A solution to the problem consists in assigning the starting times of the execution of each activity a of each project p, so that the restrictions of precedence between the activities are fulfilled and that the necessary resources are available without interruption. This problem has been tackled in different scenarios and pursuing different objectives. In this research we seek to minimize the average value of the `Makespan` added to the `Delay` of each project, which can be formulated as shown below:

$$\texttt{Min} \quad \frac{1}{\mathcal{P}}\sum_{p=1}^{\mathcal{P}}(M_p + D_p) \tag{1}$$

where M_p and D_p are the `Makespan` and `Delay` respectively associated with a p project.

978-1-7281-5666-8/19/$31.00 ©2019 IEEE

Authorized licensed use limited to: Universidad Tecnica Federico Santa Maria. Downloaded on July 30,2020 at 04:26:15 UTC from IEEE Xplore. Restrictions apply.

III. Related Work

The RCPSP is a classic combinatorial optimization problem, where there are several exact and approximate approaches to solve this problem.

Brucker et al. [7] present a B&B algorithm where they use several reference instances generated by Kolisch et al. [8], available in the PSPLIB library. In Pritsker et al. [3] present RCMPSP and propose an ILP model for resolution, followed by several exact approaches based on different mathematical modeling such as those described in [9] and [10]. Subsequently, due to the scalability and complexity of the problem, approximate approaches began to emerge. Gonçalves et al. [11] built active schedules, and parameterized based on priorities rules, delay times and release dates defined by a Genetic Algorithm (GA). Kumanan et al. [12] implemented a GA aiming at minimizing the value of TMS of the projects and [13] presented an easy and fast process of selecting priority rules for different instances, focusing on the use of GA.

In the early 2000s, the first indications of the DRCMPSP, formally introduced by Confessore et al. [4] and resolved through a multi-agent system (MAS), where each of them is responsible for local decision-making, guided by a market control mechanism coordinated by a coordinating virtual agent. Homberger [14] implements a MAS of negotiation on global resources, supported by an Evolutionary Strategy (ES). The same author in [15] proposes a MAS based on asymmetric information that agents are not willing to share, as well as their opportunistic behavior, the test instances used were the proposals in [14], in addition to a new set generated and published in the MPSPLIB library. [16] present a MAS based on auction for the distribution of global resources and critical chain to adjust the schedule and limitations of local resources of each individual project.

Some researchers have covered the DRCMPSP leaving aside the decentralized aspect of the problem. Homberger et al. [14] in addition to the proposed MAS, implement a Restart Evolution Strategy (RES) algorithm to solve RCMPSP. Wauters et al. [17] propose a learning-based optimization approach where they create a sequence of activities using a network of learning automata, focusing on achieving better overall performance. In Villafáñez et al. [18] solve the problem in a centralized way, presenting P-SGS/MIN-SLK, an algorithm based on a heuristic proposed by [19] to solve the RCPSP. These algorithms that solve the problem centrally have also been tested in MPSPLIB.

Satisfiability Modulo Theories (SMT) are a smart alternative to solve combinatorial problems, a very active area in recent times [20]. Suy et al. [21] determine that the use of SMT Solver for scheduling problems is competitive, since they have a strong Boolean component capable of expressing incompatibilities and precedence between activities.

IV. Proposal Description

Our proposal is based on a hybrid approach based on ACO and HC First Improvement. The construction of a solution consists of crossing a graph where each node $a_i p_j$ represents a i activity of a j project, and an edge $a_i p_j, a_w p_x$ represents a transition or choice of succession between the i activity of the j project towards the k activity of the p project. Each edge $a_i p_j, a_w p_x$ has an associated pheromone trail $\tau_{a_i p_j, a_w p_x}$ stored in the pheromone matrix $\mathcal{M}$ initialized at 0.1. During the execution of the algorithm if the edge $a_i p_j, a_w p_x$ was traversed in a solution, then $\tau_{a_i p_j, a_w p_x}$ is intensified following the quality of the solution. In each iteration $\tau_{a_i p_j, a_w p_x}$ is evaporated at a level ρ to avoid possible stuck in a local optimum. Each node has an associated heuristic preference $\eta_{a_i p_j}$ defined by:

$$\eta_{a_i p_j} = \frac{1}{d_{a_i p_j}} + \frac{1}{MPMT_{p_j}} \tag{2}$$

where $d_{a_i p_j}$ is the duration of the a activity of the p project and $MPMT_{p_j}$ is the time in which the project must be completed p_j to which the a_i activity belongs.

In Algorithm 1 the general procedure designed is presented.

Algorithm 1 General Procedure Pseudocode

```
1: Initialization_of_Parameters
2: S_F = ∅
3: while (N_ITER > 0) do
4:     M' = M
5:     S_Ant = ∅
6:     for i = 1 ... N_ANTS do
7:         sol = Constructive_Algorithm                    ▷ ACO
8:         if f(sol) < f(S_Ant) or S_Ant.size = 0 then
9:             S_Ant = sol
10:        end if
11:    end for
12:    S_Ant = Local_Search(S_Ant)                         ▷ HC, SMT Solver
13:    Evaporation_of_Pheromone_Levels in M
14:    if f(S_Ant) < f(S_F) then
15:        S_F = S_Ant
16:    end if
17: end while
18: return S_F
```

Initially, the number of ants N_{ANTS} and the number of iterations N_{ITER} that our algorithm will perform (line 1) is defined. To avoid that in an iteration the path of an ant influences the rest of ants, at the beginning of each iteration a copy of $\mathcal{M}$ (line 4) is made to provide information to the ants. In each iteration the ants build a solution to the problem and the best of them is stored (lines 6-11). Afterwords, LS procedure is applied to the best solution (line 12). In each iteration the pheromone evaporation is carried out in all transitions (line 13). Finally, the algorithm delivers $\mathcal{S}_F$ the best solution obtained in all iterations (lines 14-15, 18).

A. Representation

The solution of the problem is stored in the memory of the ant sol represented by a triad vector that indicates the sequence in which the activities were located. Each triad is composed of an activity that belongs to a particular project, the project and the time it was located, as shown below:

$$sol = \{ (a_1, p_j, t_1), \ldots, (a_{|A_p|}, p_{|P|}, t_M) \}$$

Authorized licensed use limited to: Universidad Tecnica Federico Santa Maria. Downloaded on July 30,2020 at 04:26:15 UTC from IEEE Xplore. Restrictions apply.

B. Constructs Solutions algorithm

In Algorithm 2 each ant builds a feasible solution to the problem.

Algorithm 2 Constructive_Algorithm (ACO) Pseudocode

1: $sol = \varnothing$
2: **for** $i = 1 \dots N_{STEPS}$ **do**
3: $\mathcal{F}$ = Check_constraints
4: $a_i p_j$ = Obtain (sol)
5: **for** $a_w p_x$ **in** $\mathcal{F}$ **do**
6: Find $\mathcal{T}_{a_i p_j, a_w p_x}$ in $\mathcal{M}'$
7: Calculate $\eta_{a_w p_x}$
8: Calculate $P_{a_i p_j, a_w p_x}$
9: **end for**
10: Select node to realize movement
11: Add the selected node to sol
12: Update resource availability
13: **end for**
14: Deposit pheromone in $\mathcal{M}$ for movements involved in sol
15: **return** sol

To build a solution the ant must make N_{STEPS} transitions or steps, which corresponds to the total activities of all projects (line 2). In each step the constraints are checked, obtaining a set $\mathcal{F}$ of nodes $a_l p_h$ towards which it is feasible to move (line 3) and the current node $a_i p_j$ where the ant is positioned is obtained (line 4). For each element of $\mathcal{F}$, the probability $P_{a_i p_j, a_w p_x}$ of the node $a_w p_x$ is selected to make the transition $a_i p_j, a_w p_x$ (lines 5-9). Subsequently, following the Roulette-Wheel selection method [22] a node is selected to be stored in sol along with the corresponding time (line 10-11), updating the amount of resources available for the next step (line 12). The value of $P_{a_i p_j, a_w p_x}$ is calculated through the equation 3.

- α : influence of memorial information (pheromone trail)
- β : influence of heuristic information

$$P_{a_i p_j, a_w p_x} = \frac{[\tau_{a_i p_j, a_w p_x}]^{\alpha} * [\eta_{a_w p_x}]^{\beta}}{\sum_{a_l p_h \in \mathcal{F}} [\tau_{a_i p_j, a_l p_h}]^{\alpha} * [\eta_{a_l p_h}]^{\beta}} \tag{3}$$

Once the solution of an ant is completed, pheromone is deposited on the path traveled according to the quality of the solution found (equation 5) (line 14).

C. Local Search algorithm

The LS is based on a Hill-Climbing algorithm with First Improvement. The HC algorithm works on feasible solutions and to verify the satisfacibility of movements, defined in Definition 1, we use the advantages of Z3 API in C++ (SMT Solver). The LS algorithm is shown in Algorithm 3.

Definition 1 (Randomized Movement). *A node is randomly selected from the $\mathcal{S}_{Ant}$, $a_i p_j = random(\mathcal{S}_{Ant})$ and $\mathcal{S}_{Ant} = \mathcal{S}_{Ant} \setminus a_i p_j$, and $a_i p_j$ is inserted in another random position $p = random(\mathcal{S}_{Ant})$ and $\mathcal{S}'_{Ant} = \mathcal{S}_{Ant}[p'] \cup a_i p_j$, where $\forall_p \in |\mathcal{S}_{Ant}|, \forall_{a_i p_j} \in \mathcal{S}_{Ant}$.*

Algorithm 3 Local_Search($\mathcal{S}_{Ant}$) Pseudocode

1: **for** $i = 1 \dots N_{ITERB}$ **do**
2: $decision$ = UNSAT
3: **while** $decision$ = UNSAT **do**
4: $\mathcal{S}'_{Ant}$ = Randomized_Movement($\mathcal{S}_{Ant}$) ▷ Definition 1
5: Φ = Boolean_Formulation($\mathcal{S}'_{Ant}$) ▷ Subsection IV-D
6: $decision$ = Check(Φ)
7: **if** $decision$ = SAT **then**
8: $\mathcal{S}'_{Ant}$ = Recalculate_times($\mathcal{S}'_{Ant}$)
9: **if** $f(\mathcal{S}'_{Ant}) < f(\mathcal{S}_{Ant})$ **then**
10: $\mathcal{S}_{Ant} = \mathcal{S}'_{Ant}$
11: **end if**
12: **end if**
13: **end while**
14: **end for**
15: Deposit pheromone in $\mathcal{M}$ for movements involved in $\mathcal{S}_{Ant}$
16: **return** $\mathcal{S}_{Ant}$

The LS receives the best solution built by the ants S_{Ant} and in N_{ITERB} iterations it carries each movement (line 4). When a movement is made within S_{Ant}, its feasibility is verified. This verification process is carried out with Z3, which checks if the precedence constraints are satisfied in a new order (lines 5-6). If the verification is unsuccessful (UNSAT) then the selection process is repeated, but if the result is SAT, the start times of each $a_i p_j$ are recalculated in $\mathcal{S}_{Ant}$ considering the availability of resources (line 7-8). In each iteration the best solution obtained so far is stored in $\mathcal{S}_{Ant}$ (lines 9-10). Finally, pheromones trails are deposited on the path traveled according to the quality of $\mathcal{S}_{Ant}$ (equation 5) (line 15).

D. Movement Satisfaction Verification Process

A motivation for the verification process satisfiability of precedence between activities to be efficient is the use of Z3 within the HC algorithm. The SMT Solvers, as we mentioned above are based on SAT, this is the first problem known as NP-hard, given a set of propositional logic formulas ($\Phi = \{\varphi_1, \varphi_2, \dots, \varphi_n\}$) on Boolean variables ($x_1, x_2, \dots, x_n$) related by logical connectives ($\neg$, $\wedge$, $\vee$). In most SAT instances and applications, the formulas are presented in *Conjunctive Normal Form* (CNF), for example, $\varphi_1 = (\neg x_1 \vee x_2)$, $\varphi_2 = (\neg x_1 \vee x_2 \vee \neg x_3)$ and $\varphi_3 = (\neg x_1 \vee x_3 \vee \neg x_4)$, they represent clauses of literals joined by disjunctions, whose final objective is to determine what values must be assigned to the literals so that $\bigwedge_{i=1}^{3} \varphi_i$ is $true$ [23].

We propose the following Boolean Constraints Formulation to verify the satisfaction of a movement.

- The parameter is defined as follows:
  - k_{ap} possible position to occupy in the memory of the ant for the a activity of the p project, $\forall_a \in \{1, \dots, |A_p|\}$, $\forall_p \in \{1, \dots, |\mathcal{P}|\}$
- The decision variable is defined as follows:
  - J_{ap} occupied position in ant memory for activity a of the project p, $\forall_a \in \{1, \dots, |A_p|\}$, $\forall_p \in \{1, \dots, |\mathcal{P}|\}$

Authorized licensed use limited to: Universidad Tecnica Federico Santa Maria. Downloaded on July 30,2020 at 04:26:15 UTC from IEEE Xplore. Restrictions apply.

- Precedence Constraints:

$$\left(\bigwedge_{a=1}^{|\mathcal{P}|}\bigwedge_{p=1}^{|A_p|}\bigwedge_{q=1}^{|A_p|} J_{ap} < J_{aq}\right) \wedge \left(\bigwedge_{a=1}^{|\mathcal{P}|}\bigwedge_{p=1}^{|A_p|} J_{ap} = k_{ap}\right) \quad (4)$$

$$\forall J_{aq} \in S_{ap}$$

E. Variation of Pheromone Levels

The pheromone deposit is made on the edges composed of a_ip_j, a_wp_x provided that a_ip_j and a_wp_x are in sol and positioned consecutively $(i,\, i+1)$. The amount of pheromone to deposit depends on the quality of sol and is calculated using the equation:

$$\tau_{a_ip_j,a_wp_x} \leftarrow \tau_{a_ip_j,a_wp_x} + \frac{1}{f(sol)}\ \forall (a_ip_j)_i, (a_wp_x)_{i+1} \in sol \quad (5)$$

where $f(sol)$ is the value of the fitness function for sol, ensuring that the longer the execution and delay time of the project, the less pheromone will be distributed along the path.

The evaporation of pheromones is done on all edges a_ip_j, a_wp_x of $\mathcal{M}$ through the following equation:

$$\tau_{a_ip_j,a_wp_x} \leftarrow (1-\rho) * \tau_{a_ip_j,a_wp_x}\ \forall a_ip_j, a_wp_x \in \mathcal{M} \quad (6)$$

In addition, with the objective of avoiding stucks in local optimal pheromone levels were limited between 0 and 1.5.

V. Computational Experiments

Our algorithms was implemented in C++11 and tested on a processor Intel(R) Xeon(R) CPU E3-1220 3.10 GHz with 32GB RAM memory on CentOS Linux 7.6.18.19 system. Our approach is validated through the online validator in MPSPLIB. We made 100 run for each of the instances, using the values of the parameters indicated in Table II.

A. Problem Instances

We perform tests on a selected set of 12 instances of MPSPLIB proposed in [15]. In them, two types of renewable resources are considered: global resources $\mathcal{G}$ (shared between activities of different projects) and local resources R_p (exclusive to each activity of each project), as shown in the Table I.

TABLE I: FEATURES OF SELECTED INSTANCES

Instance	Projects	Activities	Resource
mp_j30_a2_nr4	2	30	$\langle 3:1\rangle$
mp_j30_a5_nr2	5	30	$\langle 1:3\rangle$
mp_j30_a10_nr2	10	30	$\langle 1:3\rangle$
mp_j30_a20_nr4	20	30	$\langle 3:1\rangle$
mp_j90_a2_nr3	2	90	$\langle 2:2\rangle$
mp_j90_a5_nr4	5	90	$\langle 3:1\rangle$
mp_j90_a10_nr4	10	90	$\langle 3:1\rangle$
mp_j90_a20_nr2	20	90	$\langle 1:3\rangle$
mp_j120_a2_nr5	2	120	$\langle 1:3\rangle$
mp_j120_a5_nr2	5	120	$\langle 1:3\rangle$
mp_j120_a10_nr3	10	120	$\langle 2:2\rangle$
mp_j120_a20_nr1	20	120	$\langle 2:2\rangle$

The Table I columns 2-4 shows the features related to the number of projects that comprise it, the number of activities corresponding to each project and the distribution of global and local resources (format $\langle \mathcal{G}{:}R_p\rangle$).

B. Parameter Analysis and Tuning

We perform a parameter tuning using ParamILS [24], taking into account the parameter values used in the literature for ACO algorithms, and the following considerations:

- Number of ants, number of iterations of the algorithm and of the LS: the higher the values of these parameters imply, the greater the possibility of finding better solutions. They have a considerable influence on runtime.
- Memoristic information: if it has a value of 0, then it converts the constructive algorithm into a Greedy.
- Heuristic information: if it takes 0 value, the algorithm quickly converges to high quality solutions.
- ρ: the pheromone levels should not be evaporated quickly to keep desirable specifically those good trails.

Finally, Table II shows the ranges of values provided to the tuner and the optimal parameter settings delivered by the tuner.

TABLE II: PARAMETER SETTING

Parameter	Description	Values	Range
N_{ITER}	Number of iterations of the algorithm	100	$[50, 100]$
N_{ITERB}	Number of iterations of the local search	80	$[50, 80]$
N_{ANTS}	Number of ants	100	$[50, 100]$
α	Memorization information	0.6	$[0.3, 1.0]$
β	Heuristic information	3.0	$[2.0, 4.0]$
ρ	Evaporation level	0.2	$[0.1, 0.5]$

VI. Results

The Table IV shows the results of the Average Project Delay (ADP), Total Makespan (TMS) and Standard Deviation of the Project Delay (DPD) obtained for the 12 selected instances.

A. Discussion

In order to analyze the contribution that the Local Search algorithm makes to our constructive solution, we obtained the values found only by our Constructive algorithm for the best solution found regarding TMS values. The results are shown in Table III.

TABLE III: COMPARISON OF COMPONENTS PERFORMANCE OF OUR APPROACH

Instance	Constructive	Local Search
mp_j30_a2_nr4	58	**56**
mp_j30_a5_nr2	83	**79**
mp_j30_a10_nr2	115	**112**
mp_j30_a20_nr4	190	**187**
mp_j90_a2_nr3	116	**114**
mp_j90_a5_nr4	134	**127**
mp_j90_a10_nr4	152	**150**
mp_j90_a20_nr2	169	**167**
mp_j120_a2_nr5	128	**120**
mp_j120_a5_nr2	180	**178**
mp_j120_a10_nr3	149	**145**
mp_j120_a20_nr1	91	**88**

Authorized licensed use limited to: Universidad Tecnica Federico Santa Maria. Downloaded on July 30,2020 at 04:26:15 UTC from IEEE Xplore. Restrictions apply.

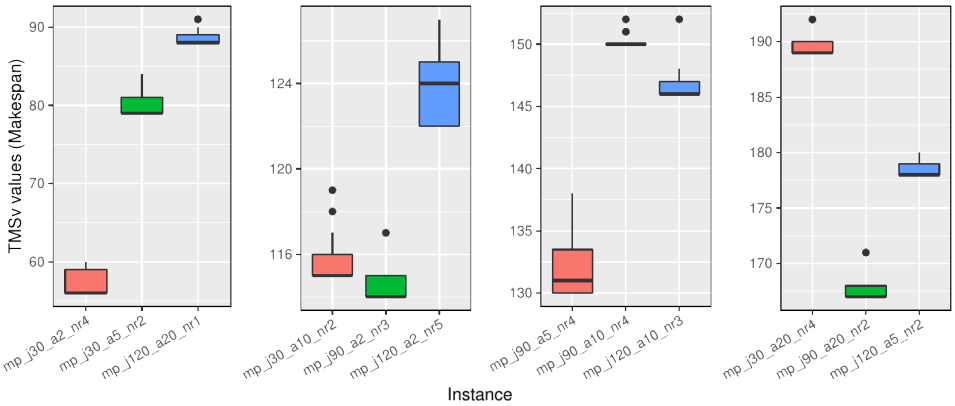

Fig. 1: Box-plots showing the TMS values (`Makespan`) for RCMPSP on MPSPLIB instances. Note: 100 runs.

TABLE IV: PERFORMANCE COMPARISON TO OTHER APPROACHES IN MPSPLIB FOR RCMPSP

	Villafáñez et al. [18]			Homberger [14]			Our approach		
Instance	APD	TMS	DPD	APD	TMS	DPD	APD	TMS	DPD
mp_j30_a2_nr4	14.5	58	9.19	14.0	59	5.65	13.0	**56**	8.48
mp_j30_a5_nr2	17.8	80	12.0	16.8	84	13.31	18.0	**79**	12.40
mp_j30_a10_nr2	13.1	114	11.64	17.2	118	14.91	17.8	**112**	8.87
mp_j30_a20_nr4	60.25	**183**	34.71	84.35	190	28.81	67.45	187	39.84
mp_j90_a2_nr3	0.5	115	0.7	0.5	**114**	0.71	6.0	**114**	8.48
mp_j90_a5_nr4	16.8	132	14.56	12.2	**123**	14.0	17.2	127	13.84
mp_j90_a10_nr4	2.1	**150**	2.33	5.1	**150**	6.36	6.2	**150**	4.87
mp_j90_a20_nr2	10.0	168	8.84	8.2	174	9.25	9.6	**167**	8.22
mp_j120_a2_nr5	9.0	118	1.41	3.0	**114**	4.24	17.0	120	7.07
mp_j120_a5_nr2	40.0	181	37.14	30.2	**178**	34.71	38.6	**178**	35.54
mp_j120_a10_nr3	12.8	146	7.39	11.5	**142**	8.54	15.3	145	9.06
mp_j120_a20_nr1	13.9	91	3.44	6.4	**82**	1.14	11.75	88	1.86

The integration of the Constructive algorithm and the Local Search algorithm always represents an improvement for the solution.

In Figure 1, we can see the boxplots with the best solutions obtained by our approach on 100 runs,based on the TMS for the selected instances. We can observe that the variability of the represented values is considerably low, so the algorithm has a stable behavior, despite the significant differences existing in the features of the selected instances.

We consider the strength of our approach is our constructive algorithm, which provides us with good quality solutions, due to the transition rule where heuristic information and pheromone trails cast by ants that have already passed through these paths are combined. In addition, the amount of pheromones to be deposited is a determining factor, because that amount is proportional to how good the solution obtained is, taking into account the value of `Makespan` and `Delay` of each project. Indeed, the smaller the `Delay` and `Makespan` of each project, the greater the amount of pheromones to deposit in each arc corresponding to the path traveled by the ant, making it more desirable for other ants. In addition, the pheromone is deposited on the best solution found in the LS procedure although there is no change with respect to the solution delivered by the constructive algorithm, increasing the desirability of this path. Furthermore, we do not allow a level of pheromone to be greater than 1.5, leading to a greater possibility of exploration for ants avoiding getting stuck on local optimum. Our way of evaluating the heuristic information allows to give priority to those activities that have a shorter duration, causing the resources to be released quickly so that other activities begin to be processed, and that the activities of those projects with less completion time, obtain a greater range amplitude within the roulette, being more likely to be selected. However, our approach consumes a lot of computational time, mainly for large instances, still taking into account that our algorithm quickly converges to good solutions this can be balanced.

B. *Comparison with best known results for RCMPSP*

The results obtained for some instances demonstrate that our approach is competitive with respect to other well-known approaches, evidenced by the ADP, TMS and DPD metrics. Taking into account that MPSPLIB library presents solutions of centralized and decentralized algorithms, we compare the quality of our solutions using the solutions proposed in the literature by Homberger in [14] with RES approach and Villafáñez et al. [18] with P-SGS/MIN-SLK approach, which present the best known results and fundamentally do not take into account the decentralized aspect of the problem.

Table IV columns 2-10 show the values for the previously described metrics found by the three approaches. It is observed that for the set of 12 instances selected, the TMS values of

Authorized licensed use limited to: Universidad Tecnica Federico Santa Maria. Downloaded on July 30,2020 at 04:26:15 UTC from IEEE Xplore. Restrictions apply.

our approach improve by 4 instances, and with the same value we find 3 solutions, while in the rest we find solutions close to the best existing ones.

Our approach finds good quality solutions, improving or matching the best known and found by centralized methods for the 12 instances in a 58% of cases. Considering that the solutions presented were selected using TMS as the main objective, for these solutions our approach delivers APD and DPD values close to those obtained by both approaches, sometimes presenting a small improvement.

The Table V shows the Friedman test, statistically comparing the results obtained by the three approaches.

TABLE V: FRIEDMAN TEST RESULTS

Approach	APD	TMS	DPD
Villafáñez et al. [18]	2.04	2.33	1.92
Homberger [14]	1.46	2.0	2.0
Our approach	2.5	1.67	2.08

According to the test, with a 95% of confidence, the results indicate that the differences between the samples are significant for the three parameters of the problem. In addition it is observed that each approach obtains better results depending on the parameter to compare, where our approach produces the lowest average range value for TMS.

VII. Conclusions and Future Work

In this work we proposed a hybrid optimization approach that integrates Ant Colony Optimization and a Local Search based on a Hill-Climbing First Improvement algorithm supported in Z3 to solve RCMPSP.

The computational results addressing the RCMPSP proved that the proposed approach was able to find better or equal solutions for some of the selected instances with respect to those known in MPSPLIB. The use of ACO proves to be a feasible resolution method to cover RCMPSP, and confirms that the algorithms belonging to swarm intelligence represent a potential solution to the problem being solved. An additional Local Search algorithm based on HC and based on Z3 was used, achieving improved solutions due to the efficiency of the process of satisfying precedence between activities in the movements. The parameter tuning process using ParamILS allowed our approach to find better values of `Makespan`.

Finally, as future work, it is essential to test our approach with the remaining 128 MPSPLIB instances and try to improve the existing results to the centralized problem. In addition, we want to model RCMPSP using the advantages offered by the Satisfiability Modulo Theories like Z3 based on approaches, techniques and models such as those proposed by [21].

Acknowledgment

This work has been partially supported by Programa de Incentivos a la Iniciación Científica (PIIC) and DGIIP of the Universidad Técnica Federico Santa María and by CONICYT-PFCHA (Doctorado Nacional/2017-21171857).

References

[1] E. Davis and J. Patterson, "An exact algorithm for the multiple constrained project scheduling problem," *Yale University*, 1969.

[2] J. K. Lenstra and A. Rinnooy Kan, "Complexity of scheduling under precedence constraints," *Operations Research*, vol. 26, no. 1, pp. 22–35, 1978.

[3] A. A. B. Pritsker, L. J. Waiters, and P. M. Wolfe, "Multiproject scheduling with limited resources: A zero-one programming approach," *Management science*, vol. 16, no. 1, pp. 93–108, 1969.

[4] G. Confessore, S. Giordani, and S. Rismondo, "A market-based multi-agent system model for decentralized multi-project scheduling," *Annals of Operations Research*, vol. 150, no. 1, pp. 115–135, 2007.

[5] M. Dorigo and M. Birattari, *Ant colony optimization*. Springer, 2010.

[6] L. De Moura and N. Bjørner, "Z3: An efficient smt solver," in *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2008, pp. 337–340.

[7] P. Brucker, S. Knust, A. Schoo, and O. Thiele, "A branch and bound algorithm for the resource-constrained project scheduling problem," *European journal of operational research*, vol. 107, no. 2, pp. 272–288, 1998.

[8] R. Kolisch, C. Schwindt, and A. Sprecher, "Benchmark instances for project scheduling problems," in *Project scheduling*. Springer, 1999, pp. 197–212.

[9] R. F. Deckro, E. Winkofsky, J. E. Hebert, and R. Gagnon, "A decomposition approach to multi-project scheduling," *European Journal of Operational Research*, vol. 51, no. 1, pp. 110–118, 1991.

[10] V. Y. Chen, "A 0–1 goal programming model for scheduling multiple maintenance projects at a copper mine," *European Journal of Operational Research*, vol. 76, no. 1, pp. 176–191, 1994.

[11] J. F. Gonçalves, J. J. Mendes, and M. G. Resende, "A genetic algorithm for the resource constrained multi-project scheduling problem," *European Journal of Operational Research*, vol. 189, no. 3, pp. 1171–1190, 2008.

[12] S. Kumanan, G. J. Jose, and K. Raja, "Multi-project scheduling using an heuristic and a genetic algorithm," *The International Journal of Advanced Manufacturing Technology*, vol. 31, no. 3-4, pp. 360–366, 2006.

[13] E. P. Vázquez, M. P. Calvo, and P. M. Ordóñez, "Learning process on priority rules to solve the rcmpsp," *Journal of Intelligent Manufacturing*, vol. 26, no. 1, pp. 123–138, 2015.

[14] J. Homberger, "A multi-agent system for the decentralized resource-constrained multi-project scheduling problem," *International Transactions in Operational Research*, vol. 14, no. 6, pp. 565–589, 2007.

[15] ——, "A (μ, λ)-coordination mechanism for agent-based multi-project scheduling," *OR spectrum*, vol. 34, no. 1, pp. 107–132, 2012.

[16] M. Chen and Z. Zhang, "A multi-agent system for drcmpsp based on auction and critical chain," *International Journal of Applied Decision Sciences*, vol. 9, no. 3, pp. 229–258, 2016.

[17] T. Wauters, K. Verbeeck, P. De Causmaecker, and G. V. Berghe, "A learning-based optimization approach to multi-project scheduling," *Journal of Scheduling*, vol. 18, no. 1, pp. 61–74, 2015.

[18] F. Villafáñez, D. Poza, A. López-Paredes, J. Pajares, and R. del Olmo, "A generic heuristic for multi-project scheduling problems with global and local resource constraints (rcmpsp)," *Soft Computing*, vol. 23, no. 10, pp. 3465–3479, 2019.

[19] R. Kolisch, "Serial and parallel resource-constrained project scheduling methods revisited: Theory and computation," *European Journal of Operational Research*, vol. 90, no. 2, pp. 320–333, 1996.

[20] L. De Moura and N. Bjørner, "Satisfiability modulo theories: introduction and applications," *Communications of the ACM*, vol. 54, no. 9, pp. 69–77, 2011.

[21] J. Suy Franch, "A satisfiability modulo theories approach to constraint programming," Ph.D. dissertation, Universitat de Girona, 2012.

[22] A. Lipowski and D. Lipowska, "Roulette-wheel selection via stochastic acceptance," *Physica A: Statistical Mechanics and its Applications*, vol. 391, no. 6, pp. 2193–2196, 2012.

[23] N. G. Ramirez, "A framework for autonomous generation of strategies in satisfiability modulo theories," Ph.D. dissertation, Universidad Tecnica Federico Santa Maria, 2018.

[24] F. Hutter, H. H. Hoos, K. Leyton-Brown, and T. Stützle, "Paramils: an automatic algorithm configuration framework," *Journal of Artificial Intelligence Research*, vol. 36, pp. 267–306, 2009.

Authorized licensed use limited to: Universidad Tecnica Federico Santa Maria. Downloaded on July 30,2020 at 04:26:15 UTC from IEEE Xplore. Restrictions apply.